User Account Management — API Testing Assessment

Sections A, B and C

Section A : Conceptual Questions

Q1

Why is API testing preferred when there is no UI?

Since there is no user interface to click on, we test the API directly to make sure the core logic works. It is faster, helps find bugs early before the UI is even built, and makes it much easier to automate security and performance checks.

Q2

What are the advantages of validating data using APIs before the UI is ready?

It saves a lot of time. We do not have to wait for the frontend team to finish building screens. Finding and fixing logic or data bugs now is much cheaper than fixing them later. It makes sure the brain of the application is solid before giving it a face.

Q3

An API returns a successful status code but incorrect data. How should response validation be handled?

A 200 OK is not enough. We must always check inside the actual response body. We do this by writing test assertions that verify the returned data matches what we expect, for example checking if the email is formatted correctly or if the name is right, rather than trusting the status code alone.

Q4

How do request headers and request body content affect API behavior?

Headers are like the envelope of a letter. They give the API important instructions like I want my answer in JSON format or here is my token to prove who I am. The body is the letter itself, the actual data we are sending such as a new user's username and password.

Q5

How should authentication validation be performed for a protected API?

We test both good and bad cases. First we send a request with a valid token to confirm it lets us in. Then we try sending requests with no token, an expired token, or a fake token. The API should block all these bad requests and return a 401 Unauthorized status.

Section B : Practical Application Tasks (Postman)

1. Collection Setup

Click New > Collection and name it User Account Management.

Add two folders inside the collection called Positive Tests and Negative Tests.

Go to the Collection Variables tab. Add a variable named base_url and set its value to your API address, for example https://api.example.com.

2. Retrieve Data (GET /users)

Tests Tab Script:

```
pm.test("Response is a JSON object", function () {

pm.response.to.be.json;

});


pm.test("Mandatory fields username and email are present", function () {

var jsonData = pm.response.json();

pm.expect(jsonData[0]).to.have.property("username");

pm.expect(jsonData[0]).to.have.property("email");

});
```

3. User Creation (POST /users)

Body (raw JSON):

```
{

"username": "johndoe123",

"email": "johndoe@example.com",

"password": "SecurePassword!99"

}
```

Tests Tab Script:

```
pm.test("Status code is 201 Created", function () {

pm.response.to.have.status(201);

});


pm.test("Response body contains a generated ID", function () {

var jsonData = pm.response.json();

pm.expect(jsonData).to.have.property("id");

});
```

4. Account Update (PUT /users/123)

Body (raw JSON):

```
{
```

```
"status": "active",

"phone_number": "1234567890"

}
```

Tests Tab Script:

```
pm.test("Status code is 200 OK", function () {

pm.response.to.have.status(200);

});


pm.test("Response reflects the updated values", function () {

var jsonData = pm.response.json();

pm.expect(jsonData.status).to.eql("active");

pm.expect(jsonData.phone_number).to.eql("1234567890");

});
```

5. Resource Removal (DELETE /users/123)

Tests Tab Script:

```
pm.test("Successful deletion status code returned", function () {

pm.expect(pm.response.code).to.be.oneOf([200, 202, 204]);

});
```

Follow-up step : Run the GET request using the same ID and verify you get a 404 Not Found.

Section C : Mini Capstone Project

1. Test Planning Document

Category	Details
Scope	Testing the core User Account Management API endpoints GET, POST, PUT, DELETE.
Objectives	Validate that users can be created, retrieved, updated, and deleted securely and that data integrity is maintained.
Test Data Strategy	Use dynamically generated test data such as random emails and usernames in Postman scripts to avoid duplicate data errors on repeat runs.
Authentication	Pass valid Bearer tokens in the Authorization header for protected routes. Test missing and invalid tokens to ensure security.
Risks	Database locking on concurrent updates and potential security leaks if error messages are too detailed.

Validation Techniques	Status code verification, JSON schema validation, response body data matching, and checking response times under 500ms.

2. Requirements Traceability Matrix

Req ID	Requirement	Test Case	Endpoint	Status
REQ-01	System must allow creation of new users	TC-001	POST /users	Covered
REQ-02	System must retrieve user details by ID	TC-002	GET /users	Covered
REQ-03	System must allow updating user details	TC-003	PUT /users/123	Covered
REQ-04	System must allow deletion of users	TC-004	DELETE /users/123	Covered

3. End-to-End Test Scenario — Complete User Lifecycle

Step	Action	Endpoint	Expected Result
1. Create	Send valid user data to create account	POST /users	Returns 201 Created. An id is generated and saved as an environment variable for next steps.
2. Retrieve	Fetch the user using the saved id	GET /users	Returns 200 OK. Returned email and username match what was sent in Step 1.
3. Update	Send a new phone number to update profile	PUT /users/123	Returns 200 OK. Response body shows the newly updated phone number.
4. Delete	Request to delete the user by id	DELETE /users/123	Returns 204 No Content or 200 OK.
5. Verify	Attempt to fetch the deleted user again	GET /users	Returns 404 Not Found, confirming the user is gone.

4. Test Summary Report

Report Section	Details
Executed Requests	Total 5 : POST, GET, PUT, DELETE, GET Verify
Pass / Fail Rate	100% Pass
Response Validation	All responses returned correct status codes. JSON structures matched expected schemas. Data integrity was maintained throughout the lifecycle.
Defects Identified	None in this cycle.

Release Recommendation	APPROVED. The User Account Management APIs are stable, secure, and meet all business requirements. Ready for frontend integration.